

SIMATS ENGINEERING

CO3-ASSESSMENT TOOL 1

Experiments

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

Code of Conduct:

I M.HARINI (192424011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student *Harini.m*

Criteria	Understandi ng of Concepts 2.5	Experimental Design 3	Use of Tools 2	Analysis of Results 1.5	Documenta tion 1	Total (10)
Q1						
Q2						
Q3						
Q4						

Faculty Signature

Experiments

QUESTIONS:4

Total Marks:40

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, and ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)
2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability. (BL4) (10)
3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP. (BL4) (10)
4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP. (BL4) (10)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Capture Packets During a TCP Connection Setup

Aim

To capture TCP packets during connection establishment and identify the three-way handshake packets: SYN, SYN-ACK, and ACK.

Requirements

- Computer/Laptop
- Wireshark
- Internet connection

Procedure

1. Open **Wireshark**.
2. Select the active network interface such as **Wi-Fi** or **Ethernet**.
3. Click **Start Capturing Packets**.
4. Open **Command Prompt**.
5. Type the following command: `curl https://example.com`
6. Return to Wireshark and stop the packet capture.
7. In the Wireshark display filter box, type: `tcp`
8. Press **Enter** to display only TCP packets.
9. To identify the SYN packet, use: `tcp.flags.syn == 1`
10. To identify the SYN-ACK packet, use: `tcp.flags.syn == 1 && tcp.flags.ack == 1`
11. To identify the ACK packet, use: `tcp.flags.ack == 1 && tcp.flags.syn == 0`
12. Select each packet and expand **Transmission Control Protocol** → **Flags** to observe the TCP flags.

Observation

Packet	TCP Flag	Function
--------	----------	----------

- | | | |
|---|-----------|--|
| 1 | SYN | Client requests to establish a connection |
| 2 | SYN + ACK | Server accepts the request and acknowledges it |
| 3 | ACK | Client acknowledges the server |

Explanation

- **SYN (Synchronize):** Initiates a TCP connection and synchronizes sequence numbers.
- **SYN-ACK:** The server acknowledges the client's SYN and sends its own SYN.
- **ACK (Acknowledgment):** The client acknowledges the server's response.

- After these three packets, the TCP connection is established and data transfer can begin.

Result

The TCP three-way handshake was successfully captured using Wireshark. The **SYN, SYN-ACK, and ACK** packets were identified, demonstrating how TCP establishes a reliable connection.

2. Capture a DNS Query Using UDP

Aim

To capture a DNS query using UDP in Wireshark and compare the UDP header with the TCP header.

Procedure

1. Open Wireshark.
2. Select the active network interface.
3. Start packet capture.
4. Open **Command Prompt**.
5. Type: nslookup google.com
6. Return to Wireshark and stop the capture.
7. Apply the following filter: dns && udp
8. Select a DNS packet.
9. Expand **User Datagram Protocol**.
10. Observe the UDP header fields.

Observation

The UDP header contains:

- Source Port
- Destination Port
- Length
- Checksum

The UDP header size is **8 bytes**.

UDP and TCP Comparison

Feature	UDP	TCP
Header size	8 bytes	Minimum 20 bytes
Connection	Connectionless	Connection-oriented

Feature	UDP	TCP
Sequence number	No	Yes
Acknowledgment	No	Yes
Retransmission	No	Yes
Speed	Faster	Comparatively slower
Reliability	Lower	Higher

Explanation

UDP does not use sequence numbers, acknowledgments, or retransmissions. Therefore, it has less overhead and can transmit data faster. However, if a UDP packet is lost, UDP does not automatically retransmit it.

DNS commonly uses UDP because DNS queries and responses are usually small and require quick communication.

Result

The DNS query was successfully captured using UDP. The UDP header was observed to be **8 bytes**, which is smaller than the TCP header. The lightweight design of UDP provides faster communication but does not provide TCP-like reliability.

3. Observe Packet Loss and TCP Retransmission

Aim

To observe how TCP retransmits lost packets and compare its behavior with UDP when packet loss occurs.

Procedure

1. Open Wireshark.
2. Select the active network interface.
3. Start packet capture.
4. Apply the filter: tcp
5. Generate TCP traffic using: curl https://example.com
6. During the communication, briefly interrupt the network connection and reconnect it.
7. Stop the capture after the connection is restored.
8. To identify TCP retransmissions, apply: tcp.analysis.retransmission

9. Observe packets marked as **TCP Retransmission**.

Observation

When a TCP packet is lost, TCP detects the missing data using acknowledgments, sequence numbers, duplicate acknowledgments, or a timeout.

The lost packet is retransmitted.

Example:

Packet 1 → Received

Packet 2 → Lost

Packet 3 → Received

Packet 2 → Retransmitted

UDP does not provide retransmission at the transport layer.

TCP vs UDP During Packet Loss

Feature	TCP	UDP
Detects lost data	Yes	No
Retransmits lost data	Yes	No
Acknowledgment	Yes	No
Sequence numbers	Yes	No
Reliability	High	Lower
Speed	Comparatively slower	Faster

Explanation

TCP provides reliable communication by detecting lost packets and retransmitting them. This increases reliability but also introduces additional delay and overhead.

UDP does not retransmit lost packets. This makes UDP faster and suitable for applications where speed is more important than guaranteed delivery.

Result

TCP retransmission was observed in Wireshark using the filter `tcp.analysis.retransmission`. TCP provides reliable delivery through acknowledgments and retransmission, whereas UDP does not provide transport-layer retransmission.

4. Capture DNS Traffic and Identify Query Types

Aim

To capture DNS traffic, identify **A**, **AAAA**, and **MX** query types, observe the responses, and measure DNS resolution time.

Procedure

1. Open **Command Prompt**.
2. Clear the DNS cache by typing: `ipconfig /flushdns`
3. Open Wireshark and select the active network interface.
4. Start packet capture.
5. Open Command Prompt and execute:
For an A record: `nslookup -type=A google.com`
For an AAAA record: `nslookup -type=AAAA google.com`
For an MX record: `nslookup -type=MX google.com`
6. Return to Wireshark and stop the capture.
7. Apply the filter: `dns`
8. Select the DNS query packet.
9. Expand **Domain Name System** → **Queries**.
10. Observe the query type and the corresponding response.

DNS Query Types

Query Type Purpose

A	Finds the IPv4 address of a domain
AAAA	Finds the IPv6 address of a domain
MX	Finds the mail server of a domain

Example

For:

```
nslookup -type=A google.com
```

The DNS server returns an **IPv4 address**.

For:

```
nslookup -type=AAAA google.com
```

The DNS server returns an **IPv6 address**.

For:

```
nslookup -type=MX google.com
```

The DNS server returns the **mail server information**.

Measuring DNS Resolution Time

In Wireshark, note the time of the DNS query and the corresponding DNS response.

Use:

Resolution Time = Response Time – Query Time

For example:

Query time = 10.123456 s

Response time = 10.145678 s

Resolution time = 10.145678 – 10.123456

= 0.022222 s

= 22.222 ms

The actual value will depend on the network and DNS server used.

Why DNS Commonly Uses UDP

DNS commonly uses UDP because:

- DNS queries are usually small.
- UDP has a small 8-byte header.
- No connection establishment is required.
- It provides faster communication with less overhead.

However, DNS can also use TCP when required, such as for some large responses and DNS zone transfers.

Result

DNS traffic was successfully captured using Wireshark. The **A**, **AAAA**, and **MX** query types were identified along with their responses. DNS commonly uses UDP because it provides fast communication with low overhead.